
AWS AppSync

AWS AppSync is a **fully managed GraphQL service** that simplifies application development by securely accessing, combining, and modifying data from multiple sources (like DynamoDB, Lambda, RDS, HTTP APIs) in real-time.

Why AppSync Matters for a Solutions Architect

Reason	Benefit
 GraphQL API Aggregation	Allows querying multiple data sources in a single request
 Real-Time Data	Built-in support for subscriptions (WebSockets)
 Security	Integrated with AWS IAM, Cognito, API Key, OIDC
 Serverless & Scalable	Fully managed; scales automatically
 Cost Efficient	Pay-per-request model like Lambda

Architecture Components

Component	Description
GraphQL Schema	Defines types, queries, mutations, and subscriptions
Resolvers	Link schema operations to data sources (e.g., DynamoDB, Lambda)

Data Sources DynamoDB, Lambda, RDS (via HTTP/ Aurora Resolver), Elasticsearch (OpenSearch), etc.

Pipeline Chain multiple operations for advanced logic

Resolvers

Subscriptions Real-time updates to clients (via WebSocket)

Common Use Cases (Exam-Relevant)

1 Mobile or Web Apps with Real-Time Features

- Use AppSync to sync chat messages, collaborative data, dashboards
- Example: Chat apps, live scoreboards, real-time order tracking

2 API Gateway Alternative with GraphQL Flexibility

- Aggregate DynamoDB + Lambda + RDS using GraphQL endpoints
- Efficient front-end development with minimal API round-trips

3 Data Aggregation Layer

- Use AppSync to unify data from disparate AWS services into a single endpoint

4 Offline Support

- Client SDKs support caching and offline data sync (ideal for mobile apps)
-

Authentication Options

Method	Usage Scenario
API Key	Simple dev/testing use
IAM Roles	Fine-grained access control from AWS services
Amazon Cognito	Federated identity (OAuth, Google, Facebook)
OpenID Connect (OIDC)	Custom identity providers

Cost Consideration

- **Charged by:** Query + data modification operations + subscriptions
 - Pay-per-use model (watch out for real-time subscription costs)
 - Great fit when scaling read-heavy, mobile-driven apps
-

Key Design Considerations (for Exam)

Choose AppSync when:

- You need **real-time updates** in a mobile/web application
- You want to **unify multiple data sources** into one endpoint
- You require **offline data sync** support (e.g., mobile apps)
- You prefer GraphQL over REST APIs

✗ Avoid AppSync when:

- You need **high-throughput backend-only ingestion** (better with Kinesis/SQS)
- You want **simple REST APIs** without schema overhead (API Gateway + Lambda is better)

📌 Comparison with API Gateway

Feature	AppSync	API Gateway
Protocol	GraphQL	REST/HTTP
Data Aggregation	Built-in	Requires client-side or Lambda orchestration
Real-Time	Supported (subscriptions)	Not supported
Offline Support	Yes (via SDKs)	No
Use Case	Mobile/web apps needing real-time sync	Backend APIs, microservices gateway

Hands-On Lab: AWS AppSync + DynamoDB + Lambda Integration

Goal:

Create a serverless GraphQL API using AWS AppSync that interacts with a DynamoDB table and AWS Lambda function.

Architecture Overview:

- **AppSync:** Exposes GraphQL API
 - **DynamoDB:** Stores product/order data
 - **Lambda:** Handles complex business logic (e.g., validation)
 - **IAM:** Secures access between services
-

Step-by-Step Instructions

Step 1: Create DynamoDB Table

1. Go to **AWS Console > DynamoDB > Create Table**
2. Table name: **Products**
3. Primary key: **id** (String)
4. Create table (leave defaults)

✅ Step 2: Create Lambda Function

1. Go to **AWS Console > Lambda > Create Function**
2. Name: **ProductProcessor**
3. Runtime: Python or Node.js
4. Sample Code (Node.js):

```
exports.handler = async (event) => {  
  
  const { id, name, price } = event.arguments.input;  
  
  return {  
  
    id,  
  
    name,  
  
    price,  
  
    status: "Processed"  
  
  };  
};
```

5. Deploy function

✅ Step 3: Create AppSync API

1. Go to **AWS Console > AppSync > Create API**

2. Choose: **Custom Schema > Start from scratch**

3. API name: **ProductAPI**

4. Click **Create**

✅ Step 4: Define GraphQL Schema

Go to **Schema tab**, replace with:

```
type Product {
```

```
  id: ID!
```

```
  name: String!
```

```
  price: Float!
```

```
  status: String
```

```
}
```

```
input ProductInput {
```

```
  id: ID!
```

```
  name: String!
```

```
  price: Float!
```

```
}
```

```
type Mutation {
```

```
  addProduct(input: ProductInput!): Product
```

```
}
```

```
type Query {  
  getProduct(id: ID!): Product  
}
```

Click **Save Schema**.

✓ Step 5: Attach Data Sources

1. Go to **Data Sources > Add Data Source**
2. Name: **ProductDynamoDB**
 - Type: DynamoDB
 - Table: **Products**
 - IAM Role: Create new or select existing
3. Add another Data Source:
 - Name: **LambdaProcessor**
 - Type: AWS Lambda
 - Function: **ProductProcessor**

✓ Step 6: Create Resolvers

Go to **Schema tab > Resolvers**

Attach Resolver to **Mutation.addProduct**:

1. Select **addProduct**
2. Choose **LambdaProcessor** as Data Source
3. Save Resolver

Attach Resolver to **Query.getProduct**:

1. Select **getProduct**
2. Choose **ProductDynamoDB**
3. Request Mapping Template:

```
{  
  
  "version": "2018-05-29",  
  
  "operation": "GetItem",  
  
  "key": {  
  
    "id": $util.dynamodb.toDynamoDBJson($ctx.args.id)  
  
  }  
  
}
```

4. Response Mapping Template:

```
$util.toJson($ctx.result)
```

✅ Step 7: Test the API

1. Go to **Queries tab**

2. Sample Mutation:

```
mutation AddProduct {  
  
  addProduct(input: {id: "101", name: "Laptop", price: 599.99}) {  
  
    id  
  
    name  
  
    price  
  
    status  
  
  }  
}
```

3. Sample Query:

```
query GetProduct {  
  
  getProduct(id: "101") {  
  
    id  
  
    name  
  
    price  
  
  }  
}
```

```
}
```

Expected Outcome

- Lambda handles input and returns status
 - DynamoDB holds product record
 - AppSync GraphQL API queries data and processes mutations
-